

UD7 - APIs REST con Laravel

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

En esta última unidad ya damos el salto a lo que se ve hoy en día en la vida real: APIs REST construidas con **Laravel**. Hasta ahora todo lo que habíamos hecho devolvía HTML al navegador, pero aquí cambiamos el chip: el servidor devuelve **JSON** y el frontend (o cualquier cliente, como Thunder Client o Postman) lo consume.

Los temas que se tocan en esta unidad:

- Cómo se monta una API REST en Laravel con `php artisan install:api` y rutas en `routes/api.php`.
- API Resources para controlar exactamente qué se devuelve.
- **Middlewares** para proteger rutas (autorización por rol, autenticación, etc.).
- **Sanctum** y autenticación por tokens.
- **Testing** automatizado con PHPUnit.

Cada apartado tiene su propio proyecto Laravel para no mezclar cosas.

Índice de proyectos

Carpeta	Tema
1.API/apinote	Primera API REST: CRUD de notas con Resource
2.Custom/apicity	API Resources personalizados + ciudades
3.Middleware/middleware1	Middleware "is admin" para proteger rutas
3.Middleware/middleware2	Segundo ejemplo de middleware
4.Auth/1authcontroller	AuthController básico con Sanctum

Carpeta	Tema
4.Auth/crud-venues	CRUD completo de venues con tokens Sanctum
5.Testing/testexample	Pruebas unitarias y de feature con PHPUnit

1. API REST - 1.API/apinote

Primer proyecto Laravel orientado a API. Se crea un CRUD básico de **notas** (título + contenido), con todas las operaciones REST: **GET**, **POST**, **PUT/PATCH**, **DELETE**.

Lo más importante que aprendí en este apartado:

- Hay que ejecutar `php artisan install:api` para que se generen las rutas `api.php` y la configuración necesaria.
- El controlador se crea con `php artisan make:controller NoteController -api`. La opción `--api` evita generar los métodos `create` y `edit`, que solo tendrían sentido si se usaran vistas.
- Las rutas resource para API se declaran así:

```
Route::apiResource('/note', NoteController::class);
```

- Cuando muestro las rutas con `php artisan route:list`, todas tienen el prefijo `/api/`.

Más detalles, endpoints completos y capturas en el README del proyecto:

[1.API/apinote/README.md](#).

2. API Resources - 2.Custom/apicity

Una vez tengo claro cómo se monta una API REST, llega el siguiente paso:

Resources. Un Resource es una clase que transforma un modelo en un array, y se usa para decidir exactamente qué campos quieres exponer al exterior.

Por ejemplo, puede que no quiera mostrar el `password_hash` o el `created_at`. El Resource me permite filtrar todo eso de forma centralizada.

En este apartado se hace otro CRUD, esta vez de **ciudades** (**City**), añadiendo **HasApiTokens** al modelo **User** para preparar el terreno de la autenticación.

Detalle en: [2.Custom/apicity/README.md](#).

3. Middlewares - **3.Middleware/**

Los **middlewares** son filtros que se ejecutan antes o después de que la petición llegue al controlador. Sirven para muchas cosas: autenticación, autorización, logging, CORS, etc.

En esta unidad veo dos casos:

- **middleware1**: middleware "is admin" que comprueba si el parámetro **user** recibido (en query, header o body) es exactamente "**admin**". Si no, devuelve **403 Forbidden**.
- **middleware2**: segundo ejemplo de middleware con más variantes.

Tipos de middleware en Laravel:

- **Global**: se aplica a todas las peticiones (**bootstrap/app.php** con **web** o **api**).
- **De grupo**: se aplica a un grupo de rutas.
- **De ruta**: se aplica a una sola ruta usando un alias.

El alias se registra en **bootstrap/app.php**:

```
$middleware->alias([
    'admin.name' => EnsureUserNameIsAdmin::class,
]);
```

Y se usa en la ruta:

```
Route::middleware('admin.name')->get('/admin/ping', ...);
```

Detalle completo en: [3.Middleware/middleware1/README.md](#).

4. Autenticación - 4.Auth/

Aquí entra en juego **Laravel Sanctum**, el paquete oficial para autenticación por tokens. La idea es:

1. El usuario manda **email** + **password** al endpoint **/auth/login**.
2. El servidor devuelve un **token**.
3. En las siguientes peticiones, el cliente manda el token en el header **Authorization: Bearer <token>**.
4. El middleware **auth:sanctum** valida el token antes de pasar al controlador.

Dos proyectos aquí:

- **1authcontroller**: primera versión simple con **AuthController** propio (register, login, logout).
- **crud-venues**: CRUD completo de **venues** (locales de eventos) con autenticación Sanctum, FormRequests para validación, factories y seeders.

Detalles en:

- [4.Auth/1authcontroller/README.md](#)
- [4.Auth/crud-venues/README.md](#)

5. Testing - 5.Testing/testexample

El último apartado de la unidad: **testing automatizado** con PHPUnit. Laravel trae testing integrado de serie, con dos tipos de tests:

- **Unit tests** (**tests/Unit/**): prueban funciones o clases en aislamiento.
- **Feature tests** (**tests/Feature/**): prueban endpoints completos haciendo peticiones HTTP simuladas.

Comandos clave:

```
php artisan test # Ejecuta todos los tests
php artisan make:test ExampleTest # Crea un test
```

Cómo arrancar cualquiera de los proyectos

Todos los proyectos son aplicaciones Laravel independientes. Los pasos comunes son:

```
cd <carpeta_del_proyecto>
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve
```

Por defecto Laravel arranca en <http://127.0.0.1:8000>. Para probar la API se puede usar **Thunder Client** (extensión de VS Code), **Postman** o [curl](#).

Conclusiones de la unidad

- Construir una API en Laravel es **muy rápido** una vez entiendes el flujo: ruta → controlador → modelo → JSON de vuelta.
- Los **Resources** son clave para no exponer datos sensibles ni acoplar la API al modelo de la base de datos.
- Los **middlewares** son la herramienta natural para autorización y filtros transversales.
- **Sanctum** simplifica la autenticación por tokens muchísimo. Para SPAs y APIs móviles es el estándar de facto en Laravel.
- **Probar el código** desde el principio ahorra problemas grandes en proyectos más serios.

Esta unidad es la que más se parece a lo que se hace en una empresa real. Todo lo anterior (PHP procedural, POO, HTTP, sesiones, autenticación con SQLite...) cobra sentido al juntarlo en un framework moderno como Laravel.